

Programmation Web 1

Langage de balisage XML

Table des matières

Structure d'un document XML.....	2
Transformation d'un document XML.....	4
Gestion du XML avec PHP.....	7
Gestion du XML avec simpleXML.....	11
Gestion du XML avec Javascript.....	12

Document écrit par Stéphane Gill

Novembre 2019



Ce document est soumis à la licence Creative Commons Attribution 3.0 Canada. Permission vous est donnée de copier, modifier et redistribuer des copies de ce document, dans les conditions fixées par la licence, tant que cette note apparaît clairement.

XML (*Extensible Markup Language*) est un métalangage informatique de balisage générique qui dérive du SGML. Cette syntaxe est dite « extensible » car elle permet de définir différents espaces de noms, c'est-à-dire des langages avec chacun leur vocabulaire et leur grammaire, comme XHTML, XSLT, RSS, SVG... Elle est reconnaissable par son usage des chevrons (<, >) encadrant les balises. L'objectif initial est de faciliter l'échange automatisé de contenus complexes entre systèmes d'informations hétérogènes (interopérabilité). Avec ses outils et langages associés, une application XML respecte généralement certains principes :

- La structure d'un document XML est définie par un schéma;
- Un document XML est entièrement transformable dans un autre document XML.

Le langage XML est donc un langage qui permet de décrire des données à l'aide de balises et de règles que l'on peut personnaliser.

Structure d'un document XML

Un document XML est composé de 2 parties : le prologue et le corps. Le prologue correspond à la première ligne d'un document XML. Il donne des informations de traitement. Voici un exemple de prologue :

```
<?xml version = "1.0" encoding="UTF-8" standalone="yes" ?>
```

Le prologue est une balise unique qui commence par <?xml et qui se termine par ?>. Dans le prologue, on commence généralement par indiquer la version de XML que l'on utilise pour décrire nos données. Il existe actuellement 2 versions : 1.0 et 1.1. La seconde information encoding="UTF-8", est le jeu de caractères utilisé dans le document XML. La dernière information présente permet de savoir si le document XML est autonome ou si un autre document lui est rattaché.

Le corps d'un document XML est constitué de l'ensemble des balises qui décrivent les données. Il y a cependant une règle très importante à respecter dans la constitution du corps : une balise en paire unique doit contenir toutes les autres. Cette balise est appelée élément racine.

```
1 <racine>
2   <balise_1>texte</balise_1>
3   <balise_2>texte</balise_2>
4   <balise_3>texte</balise_3>
5 </racine>
```

Bien évidemment, le but est d'être le plus explicite possible dans le nommage des balises. Ainsi, le plus souvent, la balise racine aura pour mission de décrire ce qu'elle contient.

Exemple 1 : Historique (*log*) d'un système domotique.

```
1  <?xml version='1.0' encoding='ASCII' standalone = 'yes'?>
2  <historique>
3    <event date="2015/12/03 11:33:04">
4      <alarme>OFF</alarme>
5      <lumiere>ON</lumiere>
6      <chauffage>OFF</chauffage>
7      <temp1>20.0 C</temp1>
8      <temp2>20.0 C</temp2>
9      <alim>ON</alim>
10   </event>
11   <event date="2015/12/03 11:33:07">
12     <alarme>OFF</alarme>
13     <lumiere>OFF</lumiere>
14     <chauffage>OFF</chauffage>
15     <temp1>20.0 C</temp1>
16     <temp2>20.0 C</temp2>
17     <alim>ON</alim>
18   </event>
19 </historique>
```

À la ligne 3, on remarque que la balise `<event>` possède un attribut soit la date et l'heure de l'événement.

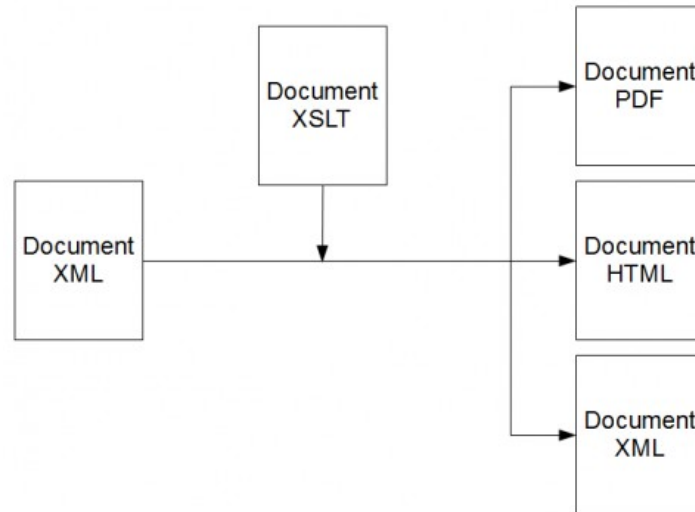
Exemple 2 : Liste de CD de musique.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <CATALOG>
3    <CD>
4      <TITLE>Empire Burlesque</TITLE>
5      <ARTIST>Bob Dylan</ARTIST>
6      <COUNTRY>USA</COUNTRY>
7      <COMPANY>Columbia</COMPANY>
8      <PRICE>10.90</PRICE>
9      <YEAR>1985</YEAR>
10   </CD>
11   <CD>
12     <TITLE>Hide your heart</TITLE>
13     <ARTIST>Bonnie Tyler</ARTIST>
14     <COUNTRY>UK</COUNTRY>
15     <COMPANY>CBS Records</COMPANY>
16     <PRICE>9.90</PRICE>
17     <YEAR>1988</YEAR>
18   </CD>
19   <CD>
20     <TITLE>Greatest Hits</TITLE>
21     <ARTIST>Dolly Parton</ARTIST>
22     <COUNTRY>USA</COUNTRY>
23     <COMPANY>RCA</COMPANY>
24     <PRICE>9.90</PRICE>
25     <YEAR>1982</YEAR>
26   </CD>
27 </CATALOG>
```

Transformation d'un document XML

XSLT (*eXtensible Stylesheet Language Transformations*) est une technologie qui permet de transformer les informations d'un document XML vers un autre type de document comme un document HTML. Comme SVG, les documents XSLT sont écrits à l'aide d'un langage de type XML.

Le principe de fonctionnement est assez simple : un document XSLT est associé à un document XML afin de créer un nouveau document d'une nature différente ou identique.



Exemple 3 : Afficher l'historique (*historique.xml*) d'un système domotique

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
3
4  <xsl:template match="/">
5    <html>
6    <link type="text/css" rel="stylesheet" href="historique.css" />
7    <body>
8    <h2>Alarme du chalet</h2>
9    <table>
10     <tr>
11       <th>Date</th>
12       <th>Alarme</th>
13       <th>Lumiere</th>
14       <th>Chauffage</th>
15       <th>Temperature 1</th>
16       <th>Temperature 2</th>
17       <th>Alimentation</th>
18     </tr>
19     <xsl:for-each select="historique/event">
20     <tr>
21       <td><xsl:value-of select="@date"/></td>
22       <td><xsl:value-of select="alarme"/></td>
```

```

23         <td><xsl:value-of select="lumiere"/></td>
24         <td><xsl:value-of select="chauffage"/></td>
25         <td><xsl:value-of select="temp1"/></td>
26         <td><xsl:value-of select="temp2"/></td>
27         <td><xsl:value-of select="alim"/></td>
28     </tr>
29 </xsl:for-each>
30 </table>
31 <p></p>
32 </body>
33 </html>
34 </xsl:template>
35 </xsl:stylesheet>

```

Le corps d'un fichier XSLT, au même titre qu'un document XML classique est constitué d'un ensemble de balises dont l'élément racine. L'élément racine d'un document XSLT est `</xsl:stylesheet>` (ligne 2 et 35). On remarque la présence de 2 attributs dans cet élément racine. Le premier est le numéro de version qui sera utilisé. Dans le cadre de ce document, c'est la version 1 qui sera utilisé. Bien qu'une version 2 existe, la première version de XSLT reste encore aujourd'hui majoritairement utilisée. Le second attribut qui est `xmlns:xsl` nous permet de déclarer un espace de noms. La déclaration de cet espace de noms, fait en sorte que toutes les balises utilisées dans un document XSLT doivent être préfixées par `xsl:`.

Le corps d'un document XSLT est composé d'un ensemble de modèle (*template*). Un modèle est défini par la balise `<xsl:template />` à laquelle plusieurs attributs peuvent être associés. Dans le cadre de cet exemple, seul l'attribut `match` est utilisé. L'attribut `match` permet de sélectionner les informations du document XML auxquelles le modèle s'applique. (s'applique à partir de la racine).

La balise `<xsl:for-each />` permet de boucler sur un ensemble d'éléments du fichier XML. Elle possède un attribut `select` permettant alors de sélectionner les informations à extraire.

La balise `<xsl:value-of />` permet d'extraire la valeur d'un élément XML ou la valeur de ses attributs. Elle possède un attribut `select` permettant alors de sélectionner les informations à extraire.

Le référencement d'un document XSLT se fait au niveau du document XML dont les informations seront utilisées au cours de la transformation. Ce référencement se fait via une petite ligne à placer sous le prologue (ligne 2) et avant l'élément racine du document XML :

```

1 <?xml version='1.0' encoding='ASCII'?>
2 <?xml-stylesheet type="text/xsl" href="historique.xsl"?>
3 <historique>
4     <event date="2015/12/03 11:33:04">
5         <alarme>OFF</alarme>
6         <lumiere>ON</lumiere>
7     ...
8 ...

```

Exemple 4 : Extraire la valeur d'un nœud.

```
1 <?xml version="1.0"?>
2 <xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
3
4 <xsl:template match="CATALOG">
5     <html>
6         <head>
7             <title>Exemple</title>
8         </head>
9         <body>
10            <p><xsl:value-of select="CD/TITLE"/></p>
11        </body>
12    </html>
13 </xsl:template>
14 </xsl:stylesheet>
```

Le modèle défini de la ligne 4 à 14, s'applique à la balise <CATALOG> du fichier XML *cd_catalog.xml*. À la ligne 10, la balise <xsl:value-of /> permet d'extraire le texte contenu entre la balise XML <TITLE> contenu dans la balise <CD>.

Exemple 5 : Appliquer des modèles

```
1 <?xml version="1.0"?>
2 <xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
3
4 <xsl:template match="CATALOG">
5     <html>
6         <head>
7             <title>Exemple</title>
8         </head>
9         <body>
10            <xsl:apply-templates/>
11        </body>
12    </html>
13 </xsl:template>
14
15 <xsl:template match="CD">
16     <p><xsl:value-of select="TITLE"/></p>
17 </xsl:template>
18
19 </xsl:stylesheet>
```

Un document XSLT est composé d'un ensemble de modèle. Chaque modèle est défini par la balise <xsl:template />. Le modèle défini aux lignes 4 à 14, s'applique à la balise XML <CATALOG>. À la ligne 11, la balise <xsl:apply-templates/> indique qu'il faut examiner tous les nœuds enfants dans l'ordre. Si un modèle qui correspond à une balise XML est détectée, il sera appliqué. Ainsi, lorsque la balise XML <CD> est détectée, le modèle aux lignes 16 à 18 est appliqué.

Exemple 6 : Parcourir les éléments d'un fichier XML

```
1 <?xml version="1.0"?>
2 <xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
3
4 <xsl:template match="CATALOG">
5     <html>
6         <head>
7             <title>Exemple</title>
8         </head>
9         <body>
10            <xsl:for-each select="CD">
11                <p><xsl:value-of select="TITLE"/></p>
12            </xsl:for-each>
13        </body>
14    </html>
15 </xsl:template>
16 </xsl:stylesheet>
```

La balise `<xsl:for-each />` de la ligne 10 permet de parcourir toutes les balises `<CD>` contenu dans le fichier XML.

Gestion du XML avec PHP

Le *Document Object Model* (DOM) est une représentation de la structure d'un document sous la forme d'une arborescence. Le DOM est une interface indépendante de tout langage de programmation ou de plate-forme. Il contient des objets, chacun représenté sous la forme d'un arbre avec ses événements associés. Il existe d'autres outils équivalents, mais la puissance de DOM réside dans sa certification par le W3C.

Grâce à l'implémentation du DOM en PHP 5, le programmeur peut créer, lire, ajouter, modifier ou supprimer les éléments d'un arbre dans un fichier XML. Cette extension du DOM est entièrement orientée objet. L'implémentation s'intègre à l'intérieur des balises PHP.

Les exemples suivants montrent comment extraire le contenu d'un fichier XML et comment créer un fichier XML.

Exemple 7 : Afficher le contenu d'un fichier XML

```
1 <?php
2     $dom = new DomDocument();
3     $dom->load("historique.xml");
4     $racine = $dom->documentElement;
5     echo "Nom de la racine: " . $racine->nodeName . "<br />";
6
7     $listeEvent = $dom->getElementsByTagName("event");
8     foreach($listeEvent as $event){
```

```

9      echo $event->getAttribute("date") . " : ";
10     echo $event->nodeValue . "<br />";
11     echo "Alarme : " . $event->getElementsByTagName('alarme')
12         ->item(0) ->firstChild->nodeValue . "<br />";
13 }
14 echo "---<br />";
15 ?>

```

À la ligne 2, un objet `DomDocument` est instancié. La variable `$dom` représente un document XML vierge, sans élément racine. Pour charger un fichier XML, la méthode `load()` (ligne 3) est utilisé.

Pour trouver des éléments, il faut d'abord récupérer l'élément racine du document (ligne 4). À la ligne 7, la variable `$listeEvent` contient la liste des nœuds `<event>` du fichier XML. La méthode `getElementsByTagName()` retourne un tableau d'objet.

Exemple 8 : Créer une liste de lecture XML

```

1  <?php
2  // On essaye de respecter le format xsfp
3  // http://www.xspf.org/quickstart/
4
5  $xml = new DomDocument("1.0", "utf-8");
6
7  // Un joli affichage
8  $xml->formatOutput = true;
9
10 // La racine
11 $listeLecture = $xml->createElement("playlist");
12 $listeLecture->setAttribute("version", "1");
13 $listeLecture->setAttribute("xmlns", "http://xspf.org/ns/0/");
14 $xml->appendChild($listeLecture);
15
16 // Titre de la playlist
17 $titre = $xml->createElement("title");
18 $listeLecture->appendChild($titre);
19 $text = $xml->createTextNode("Top 10 Rock Progressive");
20 $titre->appendChild($text) ;
21
22 // Nom de l'auteur de la playlist
23 $auteur = $xml->createElement("creator", "Dj Mad-Pug");
24 $listeLecture->appendChild($auteur);
25
26 // La liste de lecture
27 $tracklist = $xml->createElement("tracklist");
28 $listeLecture->appendChild($tracklist);
29
30 // Première pièce
31 $track = $xml->createElement("track");
32 $tracklist->appendChild($track);
33 $location = $xml->createElement("location", "file://RainingAgain.mp3");
34 $track->appendChild($location);
35 $title = $xml->createElement("title", "It's Raining Again");

```



```

36 $track->appendChild($title);
37 $creator = $xml->createElement("creator", "Supertramp");
38 $track->appendChild($creator);
39
40 $xml->save('unFichier.xml');
41 ?>

```

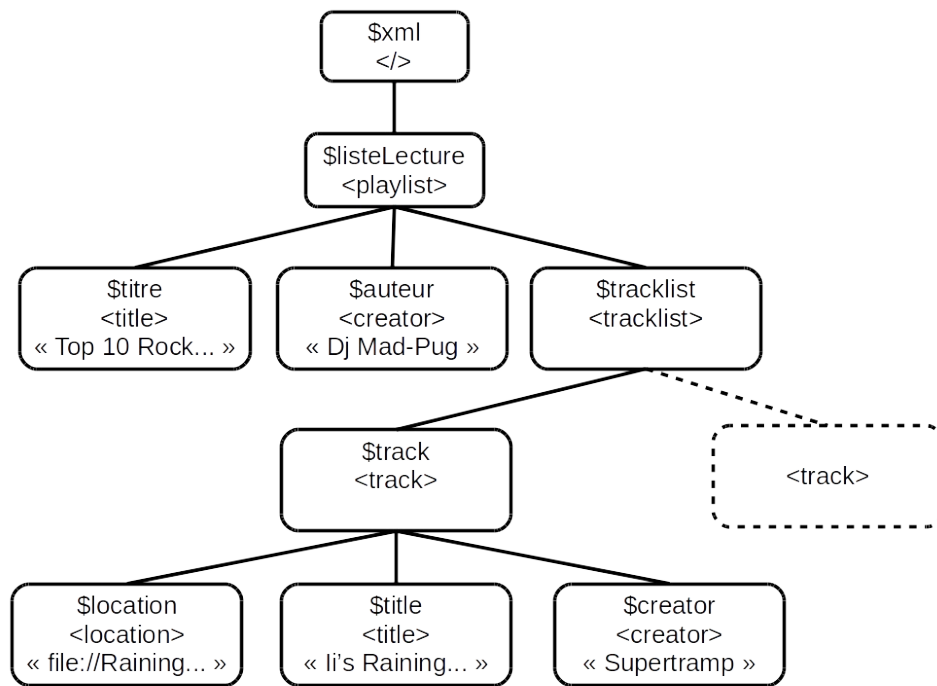
La méthode `createElement()` permet de créer des éléments XML, en passant en paramètre le nom du nœud. Par la suite, il faut ajouter un attribut au nouveau nœud en utilisant `setAttribute()`, qui sert à la fois à créer un attribut et à en modifier la valeur.

L'insertion se fait par la méthode `appendChild()`, qui ajoute le nœud passé en paramètre à la liste des enfants du nœud sur lequel il est appelé. Il sera également utile d'enregistrer le document XML sur le système de fichiers (ligne 40).

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <playlist xmlns="http://xspf.org/ns/0/" version="1">
3    <title>Top 10 Rock Progressive</title>
4    <creator>Dj Mad-Pug</creator>
5    <tracklist>
6      <track>
7        <location>file://RainingAgain.mp3</location>
8        <title>It's Raining Again</title>
9        <creator>Supertramp</creator>
10     </track>
11   </tracklist>
12 </playlist>

```



Exemple 9 : Ajouter une pièce dans une liste existante.

```

1  <?php
2  $xml = new DomDocument("1.0", "utf-8");
3  // Un joli affichage
4  $xml->preserveWhiteSpace = false;
5  $xml->formatOutput = true;
6  $xml->load("unFichier.xml");
7
8  $racine = $xml->documentElement;
9  $tracklist = $xml->getElementsByTagName("tracklist")->item(0);
10
11 // Ajouter une "track"
12 $track = $xml->createElement("track");
13 $tracklist->appendChild($track);
14 $location = $xml->createElement("location", "file://RainingAgain.mp3");
15 $track->appendChild($location);
16 $title = $xml->createElement("title", "It's Raining Again");
17 $track->appendChild($title);
18 $creator = $xml->createElement("creator", "Supertramp");
19 $track->appendChild($creator);
20 $xml->save("unAutreFichier.xml");
21 ?>

```

Exemple 10 : Charger un fichier XML et appliquer une feuille de style XSLT

```
1  <?php
2      $xslDoc = new DOMDocument();
3      $xslDoc->load("cd_catalog.xsl");
4
5      $xmlDoc = new DOMDocument();
6      $xmlDoc->load("cd_catalog.xml");
7
8      $proc = new XSLTProcessor();
9      $proc->importStylesheet($xslDoc);
10     echo $proc->transformToXML($xmlDoc);
11 ?>
```

Gestion du XML avec simpleXML

SimpleXML est une extension de PHP qui fournit des outils très simples et faciles à utiliser pour convertir du XML en un objet qui peut être manipulé avec ses propriétés et les itérateurs de tableaux.

Exemple 11 : Afficher le contenu d'un fichier XML

```
1  <!DOCTYPE html>
2  <html>
3      <?php
4          $xml = simplexml_load_file('cd_catalog.xml') or
5              die('Echec lors de l\'ouverture du fichier Recette1.xml. ');
6      ?>
7
8      <body>
9
10         <?php
11             foreach($xml->children() as $cd) {
12                 echo "<h3>" . $cd->TITLE . "</h3>";
13                 echo "<p>" . $cd->ARTIST . "; " . $cd->YEAR . "</p>";
14             }
15         ?>
16     </body>
17 </html>
```

Exemple 12 : Créer une liste de lecture XML

```
1  <?php
2      $xmlPlaylist = <<< XML
3      <playlist>
4          <title></title>
5          <creator></creator>
6          <tracklist>
7              <track>
8                  <location></location>
```

```

9         <title></title>
10        <creator></creator>
11    </track>
12 </tracklist>
13 </playlist>
14 XML;
15 $playlist = new SimpleXMLElement($xmlPlaylist);
16 $playlist->addAttribute("version", "1");
17 $playlist->addAttribute("xmlns", "http://xspf.org/ns/0/");
18 $playlist->title = "Top 10 Rock Progressive";
19 $playlist->creator = "Dj Mad-Pug";
20 $playlist->tracklist->track->location = "file://RainingAgain.mp3";
21 $playlist->tracklist->track->title = "It's Raining Again";
22 $playlist->tracklist->track->creator = "Supertramp";
23
24 Header('Content-type: text/xml');
25 $playlist->asXML("unFichier.xml");
26 ?>

```

Exemple 13 : Ajouter une pièce dans une liste de lecture

```

1  <?php
2      $playlist = simplexml_load_file('unFichier.xml')
3          or die('Echec lors de l\'ouverture du fichier.');
```

4

```

5      $playlist->tracklist->addChild("track");
6      $dernier = $playlist->tracklist->track->count() - 1;
7      $playlist->tracklist->track[$dernier]->addChild("location",
8          "file://BloodyWellRight.mp3");
9      $playlist->tracklist->track[$dernier]->addChild("title",
10         "Bloody Well Right");
11     $playlist->tracklist->track[$dernier]->addChild("creator",
12         "Supertramp");
13
14     Header('Content-type: text/xml');
15     $playlist->asXML("unAutreFichier.xml");
16 ?>

```

Gestion du XML avec Javascript

Exemple 14 : Charger un fichier XML dans une page Web

```

1  <!DOCTYPE html>
2  <html>
3
4  <head>
5      <title>Charger un fichier XML</title>
6  </head>
7
8  <body>
9

```

```

10 <h1>Charger un fichier XML</h1>
11
12 <button type="button" onclick="loadDoc()">Charger le fichier XML</button>
13
14 <table id="demo"></table>
15
16 <script>
17     function loadDoc() {
18         let url = 'cd_catalog.xml';
19         fetch(url)
20             .then(reponse => reponse.text())
21             .then(xmlText => afficherXML(xmlText));
22     }
23
24     function afficherXML(xmlText) {
25         //alert(xmlText);
26         let parser = new DOMParser();
27         let xmlDoc = parser.parseFromString(xmlText, "application/xml");
28
29         let html = "<tr><th>Artiste</th><th>Titre</th></tr>";
30         var x = xmlDoc.getElementsByTagName("CD");
31         for (let i = 0; i < x.length; i++) {
32             html += "<tr><td>" +
33                 x[i].getElementsByTagName("ARTIST")[0].childNodes[0].nodeValue +
34                 "</td><td>" +
35                 x[i].getElementsByTagName("TITLE")[0].childNodes[0].nodeValue +
36                 "</td></tr>";
37         }
38         document.getElementById("demo").innerHTML = html;
39     }
40 </script>
41
42 </body>
43
44 </html>

```

```

<?xml version="1.0" encoding="UTF-8"?>
<CATALOG>
  <CD>
    <TITLE>Empire Burlesque</TITLE>
    <ARTIST>Bob Dylan</ARTIST>
    <COUNTRY>USA</COUNTRY>
    <COMPANY>Columbia</COMPANY>
    <PRICE>10.90</PRICE>
    <YEAR>1985</YEAR>
  </CD>
  <CD>
    <TITLE>Hide your heart</TITLE>
    <ARTIST>Bonnie Tyler</ARTIST>
    <COUNTRY>UK</COUNTRY>
    <COMPANY>CBS Records</COMPANY>
    <PRICE>9.90</PRICE>
    <YEAR>1988</YEAR>
  </CD>
  ...
  ...

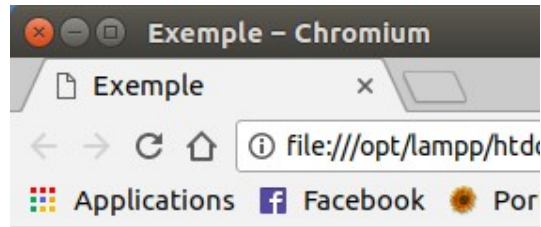
```

Annotations:

- xmlDoc.getElementsByTagName("CD")[0]
- x[0].getElementsByTagName("ARTIST")[0]
- xmlDoc.getElementsByTagName("CD")[1]
- x[1].getElementsByTagName("TITLE")[0].childNodes[0].nodeValue

Exemple 15 : Extraire des données météo d'un fichier XML.

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5    <meta charset="utf-8" />
6    <title>Charger la météo XML</title>
7  </head>
8
9  <body>
10    <h1>Exemple AJAX</h1>
11
12    <div id="meteo">
13      <p>Charger la météo ICI</p>
14    </div>
15
16    <button type="button" onclick="loadMeteo()">Charger</button>
17
18    <script>
19      function loadMeteo() {
20        const api_key = 'xxx';
21        const url = 'http://api.openweathermap.org/data/2.5/weather?
22                      q=Montreal,ca&mode=xml&units=metric&appid=' + api_key;
23
24        fetch(url)
25          .then(reponse => reponse.text())
26          .then(xmlText => {
27            let parser = new DOMParser();
28            let xmlDoc = parser.parseFromString(xmlText, "application/xml");
29            let city = xmlDoc.getElementsByTagName("city")[0];
30            let texte = "noeud : " + city.nodeName + "<br/>";
31            texte += "name : " + city.getAttribute("name") + "<br/>";
32            texte += city.childNodes[1].nodeName + " : ";
33            texte += city.childNodes[1].childNodes[0].nodeValue + "<br/>";
34
35            let temperature = xmlDoc.getElementsByTagName("temperature")[0];
36            texte += "noeud : " + temperature.nodeName + "<br/>";
37            texte += "value : " + temperature.getAttribute("value") + "<br/>";
38            texte += "unit : " + temperature.getAttribute("unit") + "<br/>";
39            document.getElementById("meteo").innerHTML = texte;
40          });
41      }
42    </script>
43  </body>
44 </html>
```



Exemple AJAX

noeud : city
 name : Montreal
 country : CA
 noeud : temperature
 value : -23.32
 unit : metric

Le contenu du fichier xml envoyé par le serveur du site *openWeatherMap* :

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <current>
3    <city id="6077243" name="Montreal">
4      <coord lon="-73.61" lat="45.5"></coord>
5      <country>CA</country>
6      <sun rise="2018-01-06T12:34:02" set="2018-01-06T21:27:22"></sun>
7    </city>
8    <temperature value="-23.32" min="-24" max="-23" unit="metric"></temperature>
9    <humidity value="63" unit="%"></humidity>
10   <pressure value="1015" unit="hPa"></pressure>
11   <wind>
12     <speed value="11.3" name="Strong breeze"></speed>
13     <gusts value="13.9"></gusts>
14     <direction value="250" code="WSW" name="West-southwest"></direction>
15   </wind>
16   <clouds value="90" name="overcast clouds"></clouds>
17   <visibility value="4828"></visibility>
18   <precipitation mode="no"></precipitation>
19   <weather number="600" value="light snow" icon="13n"></weather>
20   <lastupdate value="2018-01-06T12:00:00"></lastupdate>
21 </current>

```

Exemple 16 : Parcourir un fichier XML

```

1  <body>
2    <h1>Exemple AJAX</h1>
3
4    <div id='showCD'></div><br>
5    <input type="button" onclick="previous()" value="<<">

```

```

6      <input type="button" onclick="next()" value=">>">
7
8      <script>
9          let i = 0;
10         chargerCD(i);
11
12         function chargerCD(i) {
13             let url = 'cd_catalog.xml';
14             fetch(url)
15                 .then(reponse => reponse.text())
16                 .then(xmlText => afficherCD(xmlText, i));
17         }
18
19         function afficherCD(xmlText, i) {
20             let parser = new DOMParser();
21             let xmlDoc = parser.parseFromString(xmlText, "application/xml");
22             x = xmlDoc.getElementsByTagName("CD");
23             document.getElementById("showCD").innerHTML =
24                 "Artiste: " +
25                 x[i].getElementsByTagName("ARTIST")[0].childNodes[0].nodeValue +
26                 "<br>Titre: " +
27                 x[i].getElementsByTagName("TITLE")[0].childNodes[0].nodeValue +
28                 "<br>Annee: " +
29                 x[i].getElementsByTagName("YEAR")[0].childNodes[0].nodeValue;
30         }
31
32         function next() {
33             if (i < x.length - 1) {
34                 i++;
35                 chargerCD(i);
36             }
37         }
38
39         function previous() {
40             if (i > 0) {
41                 i--;
42                 chargerCD(i);
43             }
44         }
45     </script>
46 </body>
47 </html>

```

Exemple 17 : Charger un fichier XML et appliquer une feuille de style XSLT

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4      <script src="JS_XSLT.js"></script>
5  </head>
6  <body onload="displayResult()">
7      <div id="example" />

```



```
8 </body>
9 </html>
```

```
1 function displayResult() {
2   let xmlDoc;
3   let xslDoc;
4   let parser = new DOMParser();
5
6   fetch("cd_catalog.xml")
7     .then(reponse => response.text())
8     .then(xmlText => {
9       xmlDoc = parser.parseFromString(xmlText, "application/xml");
10      fetch("cd_catalog.xsl")
11        .then(reponse => response.text())
12        .then(xslText => {
13          xslDoc = parser.parseFromString(xslText, "application/xml");
14          xsltProcessor = new XSLTProcessor();
15          xsltProcessor.importStylesheet(xslDoc);
16          resultDocument = xsltProcessor.transformToFragment(xmlDoc, document);
17          document.getElementById("example").appendChild(resultDocument);
18        });
19    });
20 }
```